

Name :- Jaykishan J Saharan.

Div :- D Roll no. :- 26

Course/Program :- BTech CSE SY.

Subject :- DSA

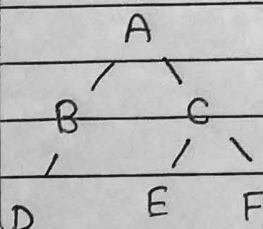
DSA theory assignment 5.

Q.1) Explain the difference between a binary tree and a binary search tree (BST). Give one example of each.

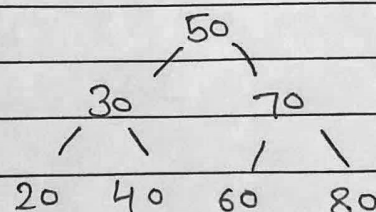
Sol :- A Binary tree is a hierarchical data structure where each node can have at most two children; typically referred to as the left child and the right child. There are no specific ordering constraints on the values within the nodes.

- A binary search tree (BST) is a special type of binary tree that maintains a specific ordering property:
- for any given node, all values in its left subtree are less than the node's own value.
 - All value in its right subtree are greater than the node's own value.
 - Both left and right subtrees must also be binary search trees.
 - Duplicate values are typically not allowed in a BST.

Example: Binary Tree



Binary Search Tree (BST)



Q.2) Write Pseudo-C++ code to create a binary search tree.
Soln:-

```
Struct Node {  
    int data;  
    Node* left;  
    Node* right;  
};
```

```
Node* createNode (int value) {  
    Node* newNode = new Node ();  
    newNode->data = value;  
    newNode->left = nullptr;  
    newNode->right = nullptr;  
    return newNode;  
}
```

```
Node* insert (Node* root, int value) {  
    if (root == nullptr) {  
        return createNode (value);  
    }  
    if (value < root->data) {  
        root->left = insert (root->left, value);  
    }  
    else if (value > root->data) {  
        root->right = insert (root->right, value);  
    }  
    return root;  
}
```

```

void inorderTraversal (Node * root) {
    if (root != nullptr) {
        inorderTraversal (root->left);
        Print (root->data);
        inorderTraversal (root->right);
    }
}

```

```

void main () {
    Node * root = nullptr;
    root = insert (root, 50);
    root = insert (root, 30);
    root = insert (root, 70);
    root = insert (root, 20);

```

```

    Print ("In-order Traversal:");
    inorderTraversal (root);
    Print ("\n");
}

```

Q.3) Construct a Binary Search Tree (BST) by inserting the elements in the given order: 50, 30, 70, 20, 40, 60, 80

Solⁿ:- The Binary Search Tree (BST) is constructed Step-by-Step by Inserting each element from the given sequence while maintaining the BST property (left child < parent, right child > parent).

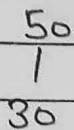
Step: i) Insert 50

Start with the first element, 50, as the root of the tree.
Root = 50

Step: 2) Insert 30

compare 30 with the root (50). Since $30 < 50$, insert it as the left child of the root.

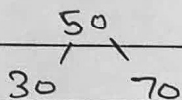
Tree:



Step: 3) Insert 70

compare 70 with the root (50). Since $50 < 70$, insert it as the right child of the root.

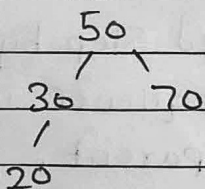
Tree:



Step: 4) Insert 20

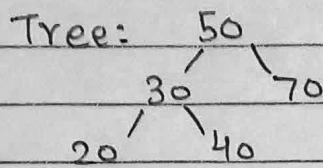
compare 20 with the root (50). Since $20 < 50$, move to the left Tree. Compare 20 with 30. Since $20 < 30$, insert it as the left child.

Tree:



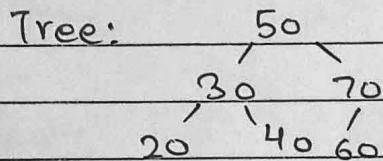
Step: 5) Insert 40

compare 40 with the root (50). Since $40 < 50$, move to the left subtree. Compare 40 with 30. Since $40 > 30$, insert it as the right child of 30.



Step: 6) Insert 60

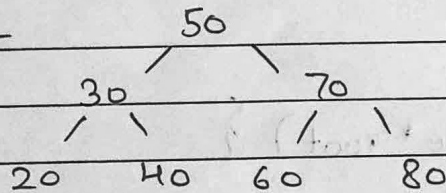
compare 60 with root (50). Since $60 > 50$, move to the right subtree. compare 60 with 70. Since $60 < 70$, insert it as the left child of 70.



Step: 7) Insert 80

compare 80 with root (50). Since $80 > 50$, move to the right subtree. compare 80 with 70. Since $80 > 70$, insert it as the right child of 70.

Answer: Tree:-



Q.4) Write Pseudo-C++ code to perform recursive preorder, inorder and postorder traversal for a binary tree.

Solⁿ:-

```

struct Node {
    int data;
    Node* left;
    Node* right;
}
  
```

```

Node (int value) {
    data = value;
    left = nullptr;
    right = nullptr;
}
};

```

```

Void PreorderTraversal (Node* root) {
    if (root == nullptr) {
        return;
    }
    Print (root->data);
    PreorderTraversal (root->left);
    PreorderTraversal (root->right);
}

```

```

Void inorderTraversal (Node* root) {
    if (root == nullptr) {
        return;
    }
    inorderTraversal (Node* root) {
        if (root == nullptr) {
            return;
        }
        inorderTraversal (root->left);
        Print (root->data);
        inorderTraversal (root->right);
    }
}

```

```

Void PostorderTraversal (Node* root) {
    if (root == nullptr) {
        return;
    }

```

```

    PostorderTraversal (root->left);

```

```

    PostorderTraversal (root->right);

```

```

    Print (root->data);

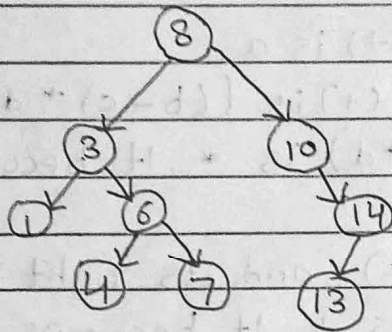
```

```

}

```

Q.5) Apply preorder, inorder and postorder traversal on the above binary search tree and write the output.
solⁿ:-



→ Preorder Traversal (Root → left → right)

Steps:- 8 → 3 → 1 → 6 → 4 → 7 → 10 → 14 → 13

output:- output:- 8, 3, 1, 6, 4, 7, 10, 14, 13

→ Inorder Traversal (left → root → right)

Steps:- 1 → 3 → 4 → 6 → 7 → 8 → 10 → 13 → 14

output:- 1, 3, 4, 6, 7, 8, 10, 13, 14

→ Postorder Traversal (left → right → root)

Steps:- 1 → 4 → 7 → 6 → 3 → 13 → 14 → 10 → 8

outputs:- 1, 4, 7, 6, 3, 13, 14, 10, 8

Q.6) Construct the expression tree for the expression $a + ((b + c) * d)$ and perform the inorder, preorder, Postorder Traversal.

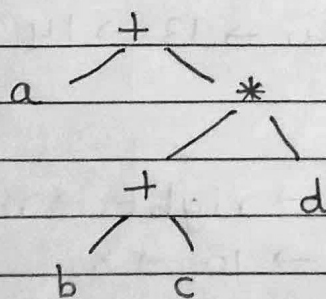
Solⁿ:-

Step:1) Construct the Expression Tree.

To construct the expression tree for $a + ((b + c) * d)$, the operators are the internal nodes and the operands are the leaf nodes. The tree is built based on the order of operations.

1. The outermost operator is $+$, which becomes the root of tree.
2. The left operand of the root $(+)$ is a .
3. The right operand of the root $(+)$ is $((b + c) * d)$.
4. The main operator in $((b + c) * d)$ is $*$. It becomes the right child of root $(+)$.
5. The left operand of $*$ is $(b + c)$, and its right operand is d .
6. The main operation in $(b + c)$ is $+$. It becomes the left child of $*$.
7. The left operand of this inner $+$ is b , and its right operand is c .

final structure of tree.



Step:2) Performing Inorder Traversal:

Inorder traversal visits the nodes in the order: left subtree, root, right subtree. This traversal method for an expression tree yields the original infix expression.

1. Traverse the left subtree of the root + (which is a).
2. Visit the root +.
3. Traverse the right subtree of root +.
4. Within the right subtree (rooted at *), traverse its left subtree (rooted at the inner +).
5. Within this subtree, traverse the left child (b), visit the root (+), and then traverse the right child (c). This yields b+c.
6. Visit the root of the right subtree (*).
7. Traverse the right child of * (which is d).
8. Combine the results to get the full traversal.

The resulting inorder Traversal is $a + b + c * d$.

Step:3) Perform Preorder Traversal

Preorder traversal visits the nodes in the order: root, left subtree, right subtree. This traversal method for an expression tree yields the prefix expression.

1. Visit the root +.
2. Traverse the left subtree of the root + (which is a).
3. Traverse the right subtree of the root + (which is *).
4. Within this subtree, visit the root *.
5. Traverse the left subtree of * (rooted at the inner +).
6. Within this subtree, visit the root +, then traverse

its left child (b), and then its right child (c). This yields $+ b c$.

7. Traverse the right child of $*$ (which is d).
8. combine the results to get the full traversal.

The resulting preorder Traversal is $+ a * + d c d$.

Step: 4) Perform Postorder Traversal.

Postorder traversal visits the nodes in the order: left subtree, right subtree, root. This traversal method for an expression tree yields the postfix expressions.

1. Traverse the left subtree of the root $+$ (which is a).
2. Traverse the right subtree of the root $+$ (rooted at $*$).
3. within this subtree, traverse its left subtree (rooted at $+$).
4. within this subtree, traverse the left child (b), then the right child (c), and finally visit the root ($+$). This yields $b c +$.
5. Traverse the right child of $*$ (which is d).
6. Visit the root of the right subtree, ($*$).
7. Visit the root $+$.
8. combine the results to get the full traversal.

The resulting postorder traversal is $a b c + d * +$.

Answer

- 1) Inorder Traversal:- $a + b + c * d$.
- 2) Preorder Traversal:- $+ a * + d c d$.
- 3) Postorder Traversal:- $a b c + d * +$.

Q.7) Write Pseudo-C++ code to perform Breadth First Search (BFS) traversal.

Solⁿ:-

```

Struct Graph {
    int v;
    Vector < Vector < int > > adjList;

```

```

Graph (int vertices) {
    v = vertices;
    adjList.resize(vertices);
}

```

```

Void addEdge (int u, int v) {
    adjList[u].push_back(v);
    adjList[v].push_back(u);
}
}

```

```

Void BFS (Graph & graph, int startVertex) {
    Vector < bool > visited (graph.v, false);
    queue < int > q;
    visited[startVertex] = true;
    q.push(startVertex);

```

```

while (!q.empty()) {
    int currentVertex = q.front();
    q.pop();
    cout << currentVertex << " ";

```

```

for (int neighbor: graph.adjList[currentVertex]) {
    if (!visited[neighbor]) {
        visited[neighbor] = true;
        q.push(neighbor);
    }
}
}
}

```

```

int main() {
    Graph g(5);
    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 3);
    g.addEdge(2, 4);
    g.addEdge(3, 4);

```

```

    cout << "BFS Traversal starting from vertex 0!";
    BFS(g, 0);
    cout << endl;

```

```

    return 0;
}

```

Q.8) Write Pseudo-c++ code to perform Depth First Search (DFS) traversal.

Solⁿ:-

```
class Graph {
private:
    int numVertices;
    list<int> * adjLists;
    bool * visited;
```

Public:

```
    Graph(int vertices) {
        numVertices = vertices;
        adjLists = new list<int>[vertices];
        visited = new bool[vertices];
        for (int i = 0; i < vertices; i++) {
            visited[i] = false;
        }
    }
```

```
};
```

```
void addEdge(int src, int dest) {
    adjLists[src].push_back(dest);
}
```

```
void DFS_util(int vertex) {
    visited[vertex] = true;
    cout << vertex << " ";
    for (int neighbor : adjLists[vertex]) {
        if (!visited[neighbor]) {
            DFS_util(neighbor);
        }
    }
}
```



```

void DFS (int startVertex) {
    for (int i=0; i<numVertices; i++) {
        visited[i] = false;
    }
    DFS_util (startVertex);
}
};

```

```

int main () {
    Graph g(5);
    g.addEdge(0,1);
    g.addEdge(0,2);
    g.addEdge(1,3);
    g.addEdge(2,4);
    g.addEdge(3,4);

```

```

    cout << "Depth first Traversal (starting from
              vertex 0): "

```

```

    g.DFS(0);
    cout << endl;

```

```

    return 0;
}

```